

Spec — Gespräche: Start Session / End Session (Single Window)

mini-moi · Guild Created: 2026-06-15 — Claude.ai Updated: 2026-06-15 — Single window, no prompt preview, clean flow Status: spec_ready For: Claude Code Depends on: Voice input + model selector already shipped

What this is

A clean Start Session / End Session flow inside the existing Gespräche page — no new tab, no prompt preview, no copy-paste. Click Start, the AI opens as the persona, Robert speaks, session ends, transcript lands in Nach der Sitzung ready for Analysieren.

Flow

1. Select persona (Maria) + scene ← existing, unchanged
2. [▶ Sitzung starten] ← NEW button, replaces "Telegram öffnen" as primary action
3. Session brief auto-POSTs to AI silently – user sees nothing, AI responds
4. Maria's opening line appears in the conversation panel (in-page)
5. [● Aufnehmen] → Robert speaks → [■ Stop] → Whisper transcribes
6. Robert's turn appears → [→ Senden] → Maria responds
7. Repeat for as many turns as needed
8. [■ Sitzung beenden]
9. Full transcript (both sides, speaker-labelled) drops into
Nach der Sitzung textarea automatically
10. Select model → [Analysieren] ← existing, unchanged

No new tabs. No prompt visible to Robert unless he wants to see it. No polling. No cross-tab state.

UI changes (Gespräche page only)

Before session starts

Current button row stays but [▶ Sitzung starten] is added as the primary/amber action:

[▶ Sitzung starten] [✈ Telegram öffnen] [📄 Kopieren] [💾 Als .txt]

Model selector sits above this row (already shipped — confirm it renders here or move it if it's currently only in Nach der Sitzung).

Active session panel (appears below buttons on start, hides on end)



- Session panel slides in below the button row when session starts
- Hides (collapses) when session ends
- Conversation scrolls within the panel — fixed height, overflow scroll
- [■ Beenden] is always visible at top right — no hunting for it

On session end

- Session panel collapses
- Full transcript auto-populates Nach der Sitzung textarea
- Status line: "Sitzung beendet ✓ — Transkript bereit"
- Scroll to Nach der Sitzung automatically

Design guard — read before touching the template

What NOT to touch

- The two-column layout (persona list left / detail right) — do not restructure
- The top navigation (mini-moi | Curator | German | Guild) — untouched
- The Mein Deutsch tab bar (Lesen | Gespräche | Schreiben | Wörter | Archiv | Admin)
- Heutige Vorbereitung section — stays exactly as-is, just scrolls below the session panel when session is active

- Nach der Sitzung section — stays exactly as-is, transcript auto-populates its existing textarea

Where the session panel inserts

The right panel currently reads top-to-bottom:

1. Persona header (emoji, name, role, description)
2. Suggested phrases (3 clickable pills)
3. Action buttons (Telegram öffnen, Kopieren, Als .txt speichern) ← add [► Sitzung starten] here
4. Heutige Vorbereitung (collapsible)
5. Nach der Sitzung (collapsible)

The session panel inserts **between items 3 and 4** when active. It does not replace anything — it inserts and pushes Heutige Vorbereitung down. `display: none` → `display: block` (or a short CSS transition — 100ms max, nothing dramatic).

```
html

<!-- existing: action buttons -->
<div class="gesprache-actions">
  <button id="start-session-btn">► Sitzung starten</button>
  <button>✈ Telegram öffnen</button>
  ...
</div>

<!-- NEW: session panel, hidden by default -->
<div id="session-panel" style="display:none" class="gesprache-session-panel">
  ...
</div>

<!-- existing: unchanged -->
<div class="heutige-vorbereitung">...</div>
<div class="nach-der-sitzung">...</div>
```

Design tokens — match exactly

Pull from the existing Gespräche stylesheet. Do not introduce new values.

Element	Value to use
Session panel background	Same parchment as the persona detail area (<code>#F5F0E8</code>) or existing var)
Panel border	Same subtle border as Heutige Vorbereitung uses
Chat log background	Slightly darker than panel — same as input fields use
Persona message text	Same as body text, normal weight
User message text	Same as body text, slightly indented or different background
[▶ Sitzung starten]	Amber/primary — same as the existing submit button in Nach der Sitzung
[■ Beenden]	Muted/secondary — same as Kopieren/Als .txt speichern
Status line	Same small muted text as the recording status line already uses
Session panel font	Georgia for the chat log (matches persona description area)

Session panel dimensions

- Width: fills the right panel content area — do not constrain it narrower
- Chat log height: fixed `280px` with `overflow-y: auto` — enough for ~4-5 turns visible, scrollable for longer sessions
- Do not set a min-height or max-height on the outer panel — let it be what it is

[▶ Sitzung starten] button placement

Sits first in the action button row, before Telegram öffnen — it's the primary action. Amber treatment so it reads as the main CTA without needing to explain itself. Same height/padding as the other buttons so the row doesn't reflow.

Active session indicator

When session is running, the persona's card in the left panel gets a subtle amber left-border (2px solid amber, same color as the accent). This mirrors how "Runde 1/5" already shows on the Maria card — extend that pattern rather than inventing a new one. Remove it on session end.

Nothing changes on mobile breakpoints

Check the existing responsive behavior of the two-column layout before adding the session

panel. The session panel should follow whatever the right column already does at narrow widths — don't add new breakpoints.

`/api/gesprache/ai-turn` (POST)

Handles both the opening turn (session brief → persona greeting) and all subsequent turns (Robert's text → persona response).

python

```
@app.route('/app/german/api/gesprache/ai-turn', methods=['POST'])
@login_required
def gesprache_ai_turn():
    data      = request.get_json()
    history   = data.get('history', []) # [{role, content}] full turn history
    persona   = data.get('persona')     # persona slug
    scene     = data.get('scene')
    model     = data.get('model', 'grok')
    user_turn = data.get('user_turn', '') # empty string on opening turn

    # First turn: history is empty, user_turn is empty
    # → send session brief as system prompt, get persona greeting
    # Subsequent turns: append user_turn to history, get next response

    response = run_chat_turn(
        history=history,
        persona=persona,
        scene=scene,
        user_turn=user_turn,
        model=model
    )
    return jsonify({'response': response, 'model': model})
```

`run_chat_turn()` in `providers/review_router.py` (extend existing module — same file, new function):

python

```
def run_chat_turn(history, persona, scene, user_turn, model):
    """
    Routes a single conversation turn to the correct provider.
    First turn (user_turn='') → persona sends greeting.
    Subsequent turns → persona responds to user_turn.
    Returns response string.
    """
    system_prompt = build_persona_system_prompt(persona, scene)
    # system_prompt is the same persona brief used in Heutige Vorbereitung
    # Claude Code: reuse existing persona prompt construction, don't rebuild

    if model == 'grok':
        return _chat_grok(system_prompt, history, user_turn)
    elif model == 'openai':
        return _chat_openai(system_prompt, history, user_turn)
    elif model == 'claude':
        return _chat_claude(system_prompt, history, user_turn)
    else:
        raise ProviderError(f'Unknown model for chat: {model}')
```

Key instruction: `build_persona_system_prompt()` must reuse the same persona brief / system prompt already used in the Heutige Vorbereitung pipeline. Don't create a new prompt — find the existing one and call it.

Frontend: session state (JS, in-page)

javascript

```

let sessionActive = false;
let sessionHistory = []; // [{role: 'user'|'assistant', content: '...'}]
let sessionTranscript = []; // ['Maria: ...', 'Sie: ...'] for final output

const startBtn = document.getElementById('start-session-btn');
const endBtn = document.getElementById('end-session-btn');
const sessionPanel = document.getElementById('session-panel');
const chatLog = document.getElementById('session-chat-log');
const sendBtn = document.getElementById('session-send-btn');
const transcriptEl = document.getElementById('gesprache-transcript'); // Nach d

// — Start Session —————

startBtn.addEventListener('click', async () => {
  sessionActive = true;
  sessionHistory = [];
  sessionTranscript = [];
  sessionPanel.style.display = 'block';
  startBtn.style.display = 'none';
  chatLog.innerHTML = '';

  // Opening turn – no user input, AI sends greeting
  await sendTurn('');
});

// — AI Turn —————

async function sendTurn(userText) {
  const model = document.getElementById('gesprache-model').value;
  const persona = document.getElementById('gesprache-persona').value;
  const scene = document.getElementById('gesprache-scene').value;

  if (userText) {
    appendMessage('Sie', userText);
    sessionHistory.push({ role: 'user', content: userText });
    sessionTranscript.push(`Sie: ${userText}`);
  }

  try {
    const res = await fetch('/app/german/api/gesprache/ai-turn', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({

```

```

        history: sessionHistory,
        persona, scene, model,
        user_turn: userText
    })
});
const data = await res.json();
const reply = data.response || '';

appendMessage(personaName(), reply);
sessionHistory.push({ role: 'assistant', content: reply });
sessionTranscript.push(`${personaName()}: ${reply}`);

} catch (err) {
    appendMessage('System', 'Fehler – bitte versuchen Sie es erneut.');
}
}

```

// — End Session —————

```

endBtn.addEventListener('click', () => {
    sessionActive = false;
    sessionPanel.style.display = 'none';
    startBtn.style.display = 'inline-block';

    // Drop transcript into Nach der Sitzung
    transcriptEl.value = sessionTranscript.join('\n');
    transcriptEl.scrollIntoView({ behavior: 'smooth' });
    document.getElementById('gesprache-status').textContent =
        'Sitzung beendet ✓ – Transkript bereit.';
});

```

// — Send button —————

```

sendBtn.addEventListener('click', async () => {
    const text = transcriptEl_session.value.trim(); // session input field
    if (!text) return;
    transcriptEl_session.value = '';
    await sendTurn(text);
});

```

// — Helpers —————

```

function appendMessage(speaker, text) {
    const div = document.createElement('div');

```

```

div.className = `chat-message chat-${speaker === 'Sie' ? 'user' : 'persona'}`
div.innerHTML = `${speaker}</strong> ${text}`;
chatLog.appendChild(div);
chatLog.scrollTop = chatLog.scrollHeight;
}

function personaName() {
  // Read from the selected persona's display name
  return document.getElementById('gesprache-persona-name').textContent || 'Pers
}

```

Voice integration: the existing Aufnehmen/Stop/Whisper flow already populates a textarea. In the session window, after Whisper transcribes, the text should auto-populate the session input field (not the Nach der Sitzung textarea), and [→ Senden] submits it as a turn. Claude Code to wire the existing transcription output to the session input field when a session is active.

What stays unchanged

- Existing [Telegram öffnen], [Kopieren], [Als .txt speichern] buttons
 - Nach der Sitzung expand/collapse
 - Analysieren with model selector
 - Whisper transcription endpoint
 - providers/review_router.py (extended, not replaced)
-

Definition of Done

- ☐ [▶ Sitzung starten] button renders, amber styling, primary action
 - ☐ Click starts session: panel slides in, AI sends opening line as persona
 - ☐ Record → Whisper → text in session input → [→ Senden] → AI responds
 - ☐ At least 3 full turns work end-to-end
 - ☐ [■ Sitzung beenden] collapses panel, transcript in Nach der Sitzung
 - ☐ Model selector works — Grok and OpenAI both respond as persona
 - ☐ Existing Telegram/copy/save buttons unaffected
 - ☐ Robert completes a full session without leaving the page or copy-pasting anything
-

Commit

bash

```
git add portal/routes/german.py portal/templates/german_gesprache.html \  
portal/providers/review_router.py  
git commit -m "feat: Gespräche Start/End Session – single window, clean flow"
```

[Start Sitzung] opens in-page session panel. Session brief auto-posts silently; AI responds as persona. Turn-by-turn: Record -> Whisper -> send -> AI responds. [End Session] collapses panel, drops full transcript into Nach der Sitzung textarea. No new tabs, no polling, no copy-paste. Extended review_router.py with run_chat_turn() for per-turn AI calls.
git push origin main

ROADMAP (not in this spec)

- Split provider chat functions into `providers/grok.py`, `providers/openai.py` as provider count grows (structural improvement, not urgent)
- Streaming AI responses (show text as it arrives)
- Session history persistence (review past conversations)

Spec · Gespräche KI-Sitzung · 2026-06-15